

Laboratorio I

a.a. 2025/2026

Correzione quarta prova in itinere

FinestraLab

Durante un'acquisizione dati in laboratorio, vogliamo mantenere una "finestra" dei campioni analizzati più recentemente (rappresentati mediante numeri interi), scartando i campioni troppo vecchi.

Si scriva una sottoclasse di Array, chiamata **FinestraLab**, con:

- una proprietà privata *maxc* (di default impostata 10), che rappresenta la lunghezza massima della finestra di campioni mantenuti nell'array.
- getter e setter, denominati *maxc*, per la proprietà *maxc*. Quando si assegna un nuovo valore *m* mediante il setter *maxc* si impone la proprietà della finestra: se *length* > *maxc* si eliminano dalla testa i campioni più vecchi finché *length* ≤ *maxc*.
- un metodo *push(e)*: funziona come *Array.push(e)*, ma se si supera *maxc*, si elimina dalla testa finché *length* ≤ *maxc*.

Per evitare operazioni ambigue: le chiamate a *pop()* e *splice()* su un'istanza di FinestraLab non devono avere nessun effetto.

Esempio

Al termine delle seguenti operazioni:

```
let fin = new FinestraLab();  
fin.push(1,6,2,3,5);  
fin.maxc = 3;
```

avremo

```
fin = [2,3,5]
```

in quanto imponendo *maxc* = 3 vengono conservati solamente gli ultimi tre campioni. Adesso, dopo l'inserimento di un ulteriore elemento:

```
fin.push(4)
```

otterremmo i seguenti 3 elementi (in quanto la finestra è al momento di dimensione *maxc*=3):

```
fin = [3,5,4]
```

FinestraLab

```
1 class FinestraLab extends Array {  
2   #pmaxc = 10;  
3  
4   get maxc() {  
5     return this.#pmaxc;  
6   }  
7  
8   set maxc(m) {  
9     while (this.length > m) super.shift();  
10    this.#pmaxc = m;  
11  }  
12  
13  push(e) {  
14    super.push(e);  
15    while (this.length > this.#pmaxc) super.shift();  
16  }  
17  
18  pop() {}  
19  splice() {}  
20 }
```

MediaMobile

Un *indicatore di media mobile* di ordine k produce una successione $a_1, a_2, \dots$ come segue.

Si parte da un numero intero positivo k , e da k valori iniziali interi: $i_1, \dots, i_k$

- I primi k termini della successione sono uguali ai valori iniziali: $a_1 = i_1, \dots, a_k = i_k$

- Per $j > k$, il j -simo termine della successione è dato dalla media dei precedenti k valori della successione, approssimata all'intero inferiore: $a_j = \left\lfloor \frac{1}{k} \sum_{i=1}^k a_{j-i} \right\rfloor$

Si scriva una classe **MediaMobile** con:

- il costruttore che prende in input k e i valori iniziali $i_1, \dots, i_k$

- un metodo generatore *succ()* che produce in ordine i termini della successione, a partire da a_1

Tutti i campi usati all'interno della classe devono essere **privati**.

Esempio

Dato $k = 3$ e valori iniziali 2, 3, 6, avremo:

- $a_1 = 2, a_2 = 3, a_3 = 6$

- $a_4 = \lfloor 1/3(a_3 + a_2 + a_1) \rfloor = \lfloor 11/3 \rfloor = 3$

- $a_5 = \lfloor 1/3(a_4 + a_3 + a_2) \rfloor = \lfloor 12/3 \rfloor = 4$

- $a_6 = \lfloor 1/3(a_5 + a_4 + a_3) \rfloor = \lfloor 13/3 \rfloor = 4$

... e così via.

MediaMobile

```
1  class MediaMobile {
2    #k;
3    #buf;
4
5    constructor(k, ... init) {
6      this.#k = k;
7      this.#buf = init.slice(0, k);
8    }
9
10   *succ() {
11     for (var i = 0; i < this.#buf.length; i++) yield this.#buf[i];
12     while (true) {
13       let s = this.#buf.reduce((a, e) => a + e, 0);
14       let n = Math.floor(s / this.#k);
15       this.#buf.shift();
16       this.#buf.push(n);
17       yield n;
18     }
19   }
20 }
```

MapStation

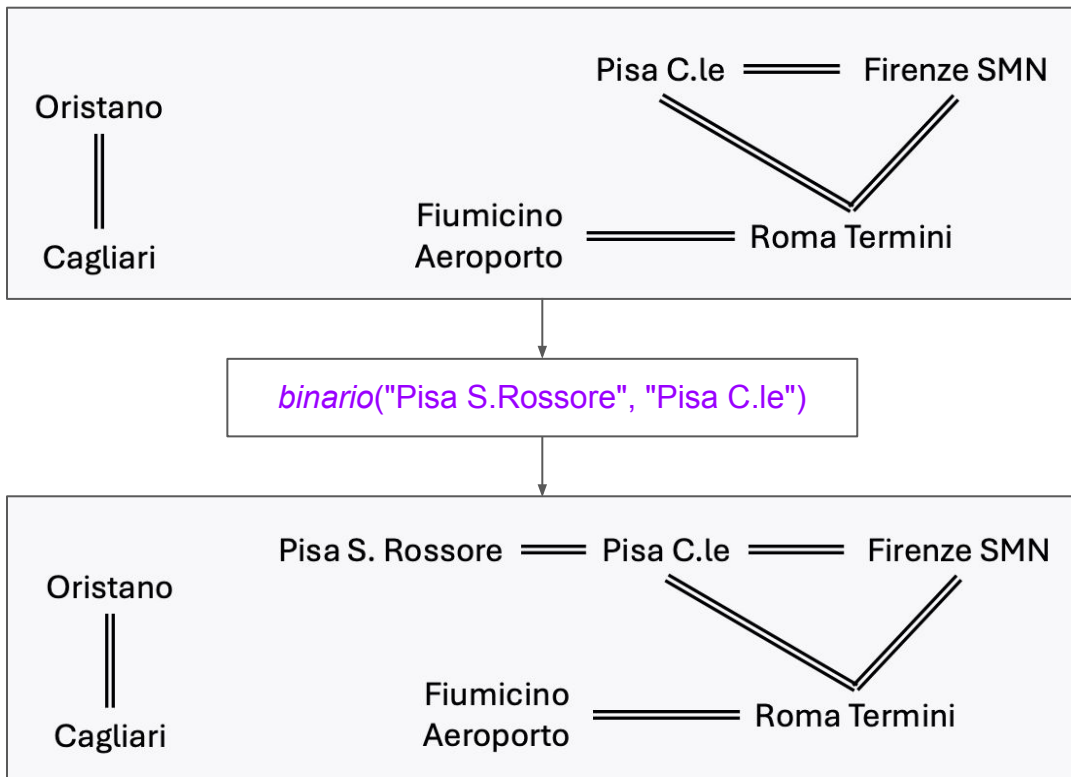
Si vuole rappresentare una mappa di collegamenti tra stazioni.

Le stazioni sono identificate da stringhe (es. "GPP", "Termini", "Pisa Centrale"). I collegamenti sono **bidirezionali**.

Si scriva una classe **MapStation** con:

- costruttore* senza argomenti
- campo pubblico *size*: contiene il numero di stazioni presenti nella mappa
- metodo *binario(u, v)*: aggiunge il collegamento tra le stazioni *u* e *v* (e crea le stazioni *u* e *v* se mancanti)
- metodo *diretto(u, v)*: restituisce true se c'è un collegamento diretto tra le stazioni *u* e *v*, false altrimenti
- metodo *raggiungibile(u, v)*: restituisce true se *v* è raggiungibile da *u*, altrimenti false
- metodo *percorso(u, v)*: restituisce un array con un cammino da *u* a *v* (**estremi inclusi**). Se non esiste, restituisce *null*.

MapStation - Esempio



`size = 6`

`diretto("Pisa C.le", "Roma Termini") = true`

`diretto("Pisa C.le", "Fiumicino Aeroporto") = false`

`raggiungibile("Pisa C.le", "Fiumicino Aeroporto") = true`

`raggiungibile("Roma Termini", "Cagliari") = false`

`percorso("Pisa C.le", "Fiumicino Aeroporto") = ["Pisa C.le", "Firenze SMN", "Roma Termini", "Fiumicino Aeroporto"]`
`, oppure ["Pisa C.le", "Roma Termini", "Fiumicino Aeroporto"]`

`percorso("Firenze SMN", "Oristano") = null`

`binario("Pisa S. Rossore", "Pisa C.le")` crea la nuova stazione **"Pisa S. Rossore"**, e aggiunge un nuovo collegamento fra **"Pisa S. Rossore"** e **"Pisa C.le"**. La `size` della mappa diventa quindi 7

MapStation - Soluzione con liste di adiacenza

```
1 class MapStation {
2   adj = {};
3   size = 0;
4
5   constructor() {}
6
7   #ensure(u) {
8     if (this.adj[u] == undefined) {
9       this.adj[u] = [];
10      this.size++;
11    }
12  }
13
14  binario(u, v) {
15    this.#ensure(u);
16    this.#ensure(v);
17    if (!this.adj[u].includes(v)) this.adj[u].push(v);
18    if (!this.adj[v].includes(u)) this.adj[v].push(u);
19  }
20
21  diretto(u, v) {
22    return this.adj[u].includes(v);
23  }
24
25  raggiungibile(u, v) {
26    return this.percorso(u, v) != null;
27  }
```

```
1 percorso(u, v) {
2   if (this.adj[u] == undefined || this.adj[v] == undefined) return null;
3
4   var seen = {};
5
6   var dfs = (x) => {
7     if (x == v) return [x];
8     seen[x] = true;
9     for (var y of this.adj[x]) {
10      if (!seen[y]) {
11        var p = dfs(y);
12        if (p != null) {
13          p.unshift(x);
14          return p;
15        }
16      }
17    }
18    return null;
19  };
20
21  return dfs(u);
22 }
```


YetAnotherAlbero - INode

Vogliamo memorizzare e cercare intervalli numerici della forma $[a, b]$, dove a e b sono numeri interi, con $a \leq b$.

Gli intervalli devono essere organizzati in un *albero binario di ricerca* (ABR) in base alla seguente regola di ordinamento:

Regola di confronto tra due intervalli $[a_1, b_1]$ e $[a_2, b_2]$:

1. Si confronta prima l'estremo sinistro, ovvero: $[a_1, b_1] < [a_2, b_2]$ se $a_1 < a_2$
2. Se gli estremi sinistri sono uguali, si confronta l'estremo destro: $[a_1, b_1] < [a_2, b_2]$ se $a_1 == a_2$ e $b_1 < b_2$

Ad esempio, $[2, 8] < [4, 5]$ (perché $2 < 4$) e $[2, 8] < [2, 12]$ (perché $8 < 12$).

Si scriva una classe **INode** che rappresenti un nodo dell'albero binario di ricerca, corrispondente ad un intervallo $[a, b]$. Tale classe deve contenere:

1. due proprietà **a,b** contenenti gli estremi dell'intervallo corrispondente ($a \leq b$)
2. la proprietà **left**, contenente il figlio sinistro del nodo (o *null* se non esiste);
3. la proprietà **right**, contenente il figlio destro del nodo (o *null* se non esiste);
4. un **costruttore** che riceve come argomento un intervallo $[a, b]$, espresso come array di due elementi
5. un metodo **add(n)** che inserisce il nodo n nel sottoalbero radicato nel nodo corrente, rispettando le regole di confronto descritte sopra.

NOTA: Potete assumere che nell'albero non sia già presente alcun nodo corrispondente allo stesso intervallo di n , in quanto gli input saranno sempre formati da intervalli diversi.

6. un metodo **findValue(x)** che, dato un numero x :

- restituisce un riferimento a un nodo il cui intervallo contiene x (cioè $a \leq x \leq b$), se esiste nel sottoalbero
- restituisce *null* se nessun intervallo nel sottoalbero contiene x

Nota: se più intervalli contengono x , è sufficiente restituirne uno qualunque.

7. un getter **maxd**, che restituisce la massima profondità del sottoalbero radicato nel nodo corrente (se il nodo corrente è una foglia, *maxd* è uguale ad 1)
8. un getter **mind** che restituisce la minima profondità del sottoalbero radicato nel nodo corrente (se il nodo corrente è una foglia, *mind* è uguale ad 1).

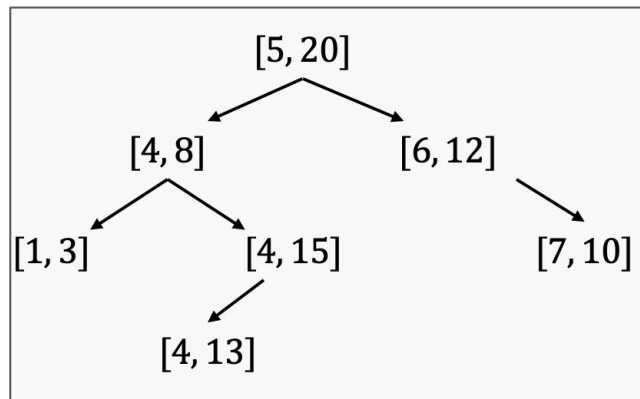
YetAnotherAlbero

Si scriva poi una classe **YetAnotherAlbero** che rappresenta l'intera struttura ABR. La classe YetAnotherAlbero deve implementare:

- un costruttore senza argomenti;
- un metodo **addInterval([a,b])** che inserisce un nuovo intervallo con estremi a , b nell'albero;
- un campo pubblico **root** che è: *null* se l'albero è vuoto; l'INode corrispondente alla radice dell'albero altrimenti
- un campo pubblico **size** che contiene il numero totale di nodi presenti nell'albero (che viene quindi aggiornato ad ogni chiamata di *addInterval()*).

YetAnotherAlbero - Esempio

Se aggiungiamo, in ordine, gli intervalli **[5,20]**, **[4,8]**, **[6,12]**, **[4,15]**, **[4,13]**, **[1,3]**, **[7,10]**, otteniamo il seguente ABR



addInterval(**[4,13]**):

- sottoalbero sx di **[5,20]** ($4 < 5$)
- sottoalbero dx di **[4,8]** ($8 < 13$),
- sottoalbero sx di **[4,15]** ($13 < 15$).

Consideriamo la radice **[5,20]**:

findValue(5) = **[5,20]** o **[4,8]** o **[4,15]** o **[4,13]**

findValue(2) = **[1,3]**

mind = **3** , *maxd* = **4**

YetAnotherAlbero

- INode

```
1 class INode {
2     left = null;
3     right = null;
4
5     constructor([a, b]) {
6         this.a = a;
7         this.b = b;
8     }
9
10    add(n) {
11        // Regola 1 & 2: confronto a, poi b
12        if (n.a < this.a || (n.a === this.a && n.b < this.b)) {
13            if (!this.left) this.left = n;
14            else this.left.add(n);
15        } else {
16            // Regola 3: duplicati o maggiori vanno a destra
17            if (!this.right) this.right = n;
18            else this.right.add(n);
19        }
20    }
21
22    findValue(x) {
23        // Caso 1: Trovato nel nodo corrente
24        if (x ≥ this.a && x ≤ this.b) return this;
25
26        // Caso 2: Se x < a, può essere SOLO a sinistra (perché a destra gli intervalli iniziano dopo a)
27        if (x < this.a) {
28            return this.left ? this.left.findValue(x) : null;
29        }
30
31        // Caso 3: Se x > a, l'intervallo potrebbe essere a destra
32        // MA anche a sinistra (un intervallo iniziato prima potrebbe finire molto dopo x)
33        let found = this.right ? this.right.findValue(x) : null;
34        if (!found) {
35            found = this.left ? this.left.findValue(x) : null;
36        }
37        return found;
38    }
}
```

YetAnotherAlbero - INode (*maxd/mind*)



```
1  get maxd() {
2      let l = this.left ? this.left.maxd : 0;
3      let r = this.right ? this.right.maxd : 0;
4      return 1 + Math.max(l, r);
5  }
6
7  get mind() {
8      // Se è una foglia, profondità 1 (come richiede il testo)
9      if (!this.left && !this.right) return 1;
10
11     // Se manca un figlio, la profondità minima del sottoalbero
12     // deve ignorare il ramo nullo per trovare la foglia più vicina
13     let l = this.left ? this.left.mind : Infinity;
14     let r = this.right ? this.right.mind : Infinity;
15     return 1 + Math.min(l, r);
16 }
```

YetAnotherAlbero



```
1  class YetAnotherAlbero {  
2      root = null;  
3      size = 0;  
4  
5      addInterval(arr) {  
6          let n = new INode(arr);  
7          if (!this.root) {  
8              this.root = n;  
9          } else {  
10             this.root.add(n);  
11         }  
12         this.size++;  
13     }  
14 }
```

Q & A